

```

// start of shader
1: let ids = num_workgroups.x * workgroup_size;
2: let base = global_invocation_id.x * 3; ← Number of patterns generated is chosen upfront. In this
    ... // other computation                      shader 3 patterns will be generated.
3: if (storage_data[0] == 0) {
4:     idx[base] = 8;
5: }
6: else {
7:     idx[base] = 11;
8: }
    ... // other computation
9: if (idx[base] < BUF_SIZE) { ← optional
    ... // other computation
10: let temp_0 = idx[base];
11: output[base].data = storage_data[temp_0];
12: output[base].idx = temp_0;
13: {}
14: idx[(((global_invocation_id.x + 8) % ids) * 3)] = (i32(global_invocation_id.x)) + BUF_SIZE;
    ... // other computation

```

Initialize idx to one of two (in-bounds) values, stopping compiler from propagating constants and using registers instead.

Perform potential out-of-bounds read and record results.

Index Values

Basic: idx[base]
 IntOverflowMult: $\text{idx[base]} * 4$
 IntOverflowAdd: $\text{idx[base]} + 512643019$
 (Div/Mod)ByZero: $511 \text{ (}\div\text{/\%)} \text{ idx[base]}$
 (Div/Mod)IntMin: $-2147483648 \text{ (}\div\text{/\%)} \text{ idx[base]}$

Race Write Values

Basic: $\text{gid.x} + \text{BUF_SIZE}$;
 IntOverflowMult: 998768123
 IntOverflowAdd: 1751312910
 (Div/Mod)ByZero: 0
 (Div/Mod)IntMin: -1

Buffer Address Spaces

Storage: storage_data
 Workgroup: workgroup_data